

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность: Программирование электронных устройств и
систем

ОТЧЕТ

по проектной практике

Студент: Церковный Матвей Алексеевич Группа: 251-329

Место прохождения практики: Московский Политех, кафедра информатика и
информационные технологии

Отчет принят с оценкой _____ Дата _____

Руководитель практики: И.В. Иванов

Москва 2026

Оглавление

ВВЕДЕНИЕ	3
Общая информация о проекте.....	3
Общая характеристика деятельности организации.....	3
Описание задания по проектной практике	4
Описание достигнутых результатов	4
БАЗОВАЯ ЧАСТЬ.....	5
Создание GitHub репозитория.....	5
Разработка статического веб-сайта.....	5
ВАРИАТИВНАЯ ЧАСТЬ	9
Реализация блокчейна	9
Модификация.....	10
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	16

ВВЕДЕНИЕ

Общая информация о проекте

Проект «Комплексная система управления умным домом».

Цели: Разработка прототипов модулей умного дома осуществляющих неинвазивный анализ биомаркеров человека и управляющих тепловым комфортом пользователей, находящихся в помещениях.

Задачи:

1. Разработка прототипа оборудования УД для осуществления оптимального управления зональным тепловым комфортом пользователей в помещениях.
2. Разработка прототипа ПО, осуществляющего оптимальное управление тепловыми актуаторами.
3. Разработка схем и 3Д моделей электронных плат изделия.
4. Разработка прототипа ПО, осуществляющего задание расчётных методик в формате электронных интерактивных стандартов.

Общая характеристика деятельности организации

Индустриальный партнер:

Общество с ограниченной ответственностью «Точка доступа».

Контактное лицо со стороны заказчика: Игнатов Евгений Александрович.

Взаимодействие с партнером заказчик формулирует задачи, выставляет сроки их реализации, предлагает методики решения задач, осуществляет руководство проектом, консультирование по выполнению задач, а также финансирование проекта.

Описание задания по проектной практике

Базовая часть задания включала в себя работу с основным проектом по дисциплине «Проектная деятельность», а именно создание статического веб-сайта и сохранение результатов в репозитории.

Вариативная часть задания включала в себя создание базовой структуры блокчейна на языке Java, включая расчет хеша через SHA-256 и базовый цикл майнинга. В качестве модификаций были добавлены транзакции, проверка авторства и P2P сеть.

Описание достигнутых результатов

В ходе выполнения проектной практики были освоены основы работы с системой контроля версий Git и платформой GitHub, разработан статический веб-сайт в рамках базовой части задания. В рамках вариативной части задания реализована базовая структура блокчейна на Java с использованием алгоритма хэширования SHA-256, разработан механизм майнинга. Также был получен опыт самостоятельного решения практических задач.

БАЗОВАЯ ЧАСТЬ

Создание GitHub репозитория

В рамках выполнения базовой части проектной практики был создан персональный репозиторий на платформе GitHub для хранения и сопровождения результатов работы. В процессе были изучены основные возможности системы контроля версий Git, включая создание репозитория, клонирование проекта, фиксацию изменений (commit), отправку данных в удалённый репозиторий (push) и организацию структуры проекта. Созданный репозиторий использовался для хранения исходного кода, документации и материалов проектной практики. Также был получен опыт работы с Markdown при оформлении файла `README.md`. Ссылка на GitHub репозиторий: <https://github.com/MatveyTserkovny/practiceTserkovnyi>

Разработка статического веб-сайта

В ходе проектной практики был разработан сайт об основном проекте по дисциплине «Проектная деятельность». Были реализованы следующие разделы:

- домашняя страница с аннотацией проекта (Рисунок 1);
- страница «О проекте» с описанием проекта (Рисунок 2);
- страница «Участники» с описанием вклада каждого участника группы в проект (Рисунок 3);
- страница «Журнал» с тремя постами о процессе работы (Рисунок 4);
- страница «Ресурсы» со ссылками на полезные материалы (Рисунок 5).

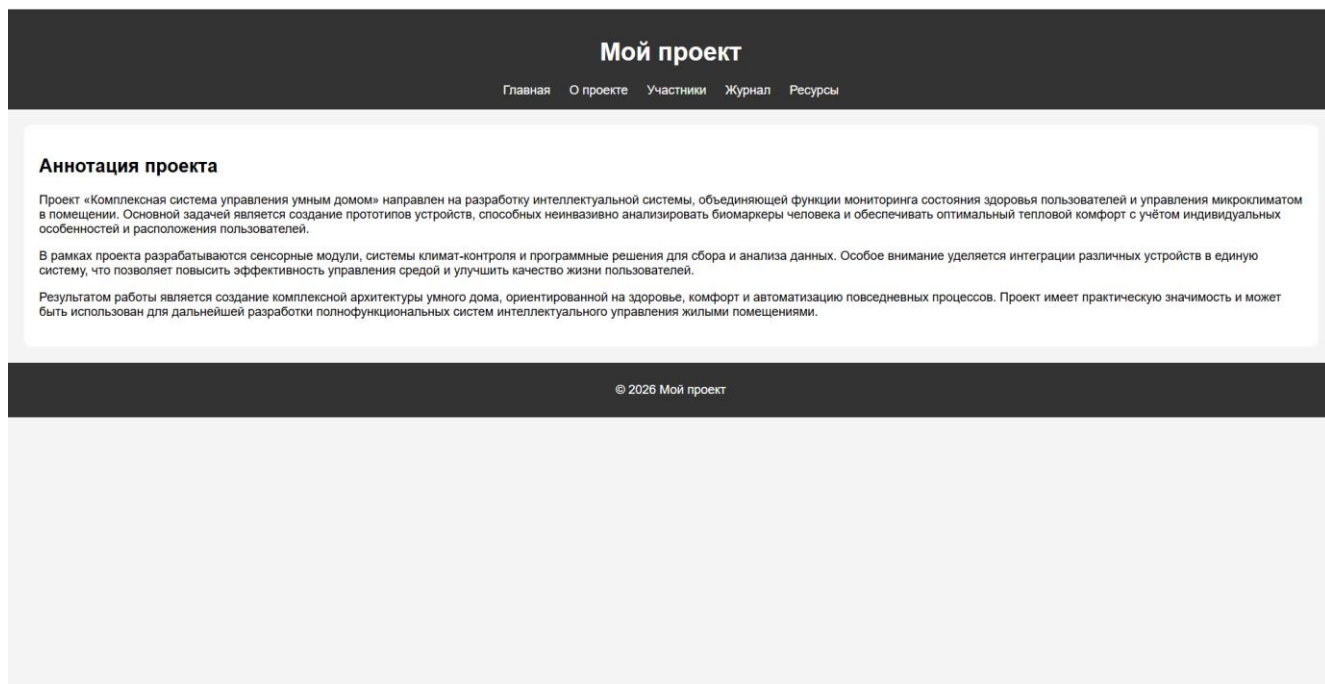


Рисунок 1 – Домашняя страница сайта



Рисунок 2 – Страница с описанием проекта

Участники

- Игнатов Евгений Александрович - контактное лицо со стороны заказчика
- Капустин Александр Андреевич - лидер проекта
- Абасов Мирза Нурмагомедович - аппаратная часть
- Айдарова Анастасия Алексеевна - аппаратная часть
- Анисковец Роман Валерьевич - аппаратная часть
- Басс Владислав Михайлович - методы оптимизации
- Бурула Александр Владимирович - методы оптимизации
- Васин Яков Евгеньевич - разработка 3D моделей корпусных элементов
- Верещагин Даниил Сергеевич - разработка 3D моделей корпусных элементов
- Воинова Екатерина Алексеевна - разработка 3D моделей корпусных элементов
- Гореликова София Евгеньевна - разработка 3D моделей корпусных элементов
- Губский Михаил Дмитриевич - методы оптимизации
- Каплан Матвей Сергеевич - методы оптимизации
- Колков Максим Дмитриевич - оформление документов
- Коригов Яхья Ахметович - аппаратная часть
- Кудашев Максим Артемович - методы оптимизации
- Митин Максим Александрович - аппаратная часть
- Овчинников Илья Дмитриевич - разработка на Avalonia
- Оприш Тимофей Васильевич - методы оптимизации
- Печеник Елизавета Владимировна - аппаратная часть
- Поповичкина Софья Денисовна - оформление документов
- Рыженков Виталий Витальевич - пишет диплом
- Сапронов Глеб Сергеевич - аппаратная часть
- Соловьев Никита Денисович - оформление документов
- Степанченко Денис Сергеевич - аппаратная часть
- Трунов Евгений Дмитриевич - аппаратная часть
- Филюшин Илья Владимирович - разработка 3D моделей корпусных элементов
- Церковный Матвей Алексеевич - прогнозирование

Рисунок 3 – Страница об участниках проекта

Журнал проекта

Задача 1. Управление тепловым комфортом

Разработана структурная схема системы зонального климат-контроля.

Созданы 3D-модели и чертежи прототипа умной заслонки-вентиля для регулировки потока воздуха.

Подобрана компонентная база (сервоприводы, датчики температуры/влажности, контроллеры) и составлены их спецификации.

Задача 2. Анализ биомаркеров

Разработана функциональная схема модуля мониторинга, определяющая состав сенсоров (оптические для пульса и SpO2, ИК-датчики для температуры кожи, радар миллиметрового диапазона для дыхания).

Созданы принципиальные электрические схемы и макеты печатных плат для сенсорного блока.

Разработаны и протестированы базовые алгоритмы первичной обработки сигналов с датчиков.

Задача 3. Сбор и анализ данных

Разработана архитектура системы сбора и анализа данных, включая методы обработки и хранения информации.

Рисунок 4 – Журнал проекта

Полезные ресурсы

- Петров А.Н. "Интеллектуальные системы управления зданиями". М.: Техносфера, 2022. 412 с.
- Козлов В.С., Сидорова Е.П. "Микроконтроллеры в системах автоматизации умного дома". СПб.: БХВ-Петербург, 2021. 356 с.
- Johnson M., Smith R. "IoT-based Health Monitoring Systems". Springer, 2023. 478 p.
- Иванов Д.К., Петрова М.С. "Алгоритмы зонального управления микроклиматом в умных зданиях" // Журнал "Автоматизация и IT в энергетике". 2023. № 5. С. 45-52.
- Chen L., Wang H. "Non-invasive Sensors for Health Monitoring in Smart Environments" // IEEE Transactions on Biomedical Engineering. 2022. Vol. 70. P. 1123-1134.
- [Официальная документация платформы Arduino](#)
- [Библиотеки для работы с датчиками DHT](#)

Рисунок 5 – Страница с полезными ресурсами

ВАРИАТИВНАЯ ЧАСТЬ

Реализация блокчейна

В вариативной части проектной практики была реализована базовая модель блокчейна на языке Java. В процессе разработки были изучены основные принципы работы распределённых систем хранения данных и криптографических алгоритмов.

Функциональность проекта включала:

- создание структуры блока;
- хранение цепочки блоков;
- вычисление SHA-256 хеша;
- связывание блоков через предыдущий хеш;
- реализацию механизма майнинга;
- проверку целостности цепочки блоков.

Разработка проекта позволила закрепить навыки объектно-ориентированного программирования и работы с криптографическими методами.

Листинг 1 – Класс Block

```
public class Block {

    public String hash;
    public String previousHash;
    public String merkleRoot;
    public ArrayList<Transaction> transactions = new ArrayList<Transaction>();
    //our data will be a simple message.
    public long timeStamp; //as number of milliseconds since 1/1/1970.
    public int nonce;

    //Block Constructor.
    public Block(String previousHash ) {
        this.previousHash = previousHash;
        this.timeStamp = new Date().getTime();

        this.hash = calculateHash(); //Making sure we do this after we set
the other values.
    }

    //Calculate new hash based on blocks contents
    public String calculateHash() {
        String calculatedhash = StringUtil.applySha256(
            previousHash +
            Long.toString(timeStamp) +
            Integer.toString(nonce) +
            merkleRoot
        );
        return calculatedhash;
    }
}
```

```

    }

    //Increases nonce value until hash target is reached.
    public void mineBlock(int difficulty) {
        merkleRoot = StringUtil.getMerkleRoot(transactions);
        String target = StringUtil.getDifficultyString(difficulty); //Create
a string with difficulty * "0"
        while(!hash.substring( 0, difficulty).equals(target)) {
            nonce ++;
            hash = calculateHash();
        }
        System.out.println("Block Mined!!! : " + hash);
    }

    //Add transactions to this block
    public boolean addTransaction(Transaction transaction) {
        //process transaction and check if valid, unless block is genesis
block then ignore.
        if(transaction == null) return false;
        if((previousHash != "0")) {
            if((transaction.processTransaction() != true)) {
                System.out.println("Transaction failed to process.
Discarded.");
                return false;
            }
        }
        transactions.add(transaction);
        System.out.println("Transaction Successfully added to Block");
        return true;
    }
}

```

```

Trying to Mine block 1...
Block Mined!!! : 00000731a61c365f093fcbab8741d2ea29191c1da408dfcdd9332b568fe38cce
Trying to Mine block 2...
Block Mined!!! : 000003dae8a626c87dbadb34325a8b272a52e3ce45e4da2c2959eb81a0c8cb9a
Trying to Mine block 3...
Block Mined!!! : 000001c813a29475aed4d4e9dd1af2f7530c177f1e3f0bcfd944cb2ec3d96e8

Blockchain is Valid: true

The block chain:
[
  {
    "hash": "00000731a61c365f093fcbab8741d2ea29191c1da408dfcdd9332b568fe38cce",
    "previousHash": "0",
    "data": "Hi im the first block",
    "timeStamp": 1513418517793,
    "nonce": 1453771
  },
  {
    "hash": "000003dae8a626c87dbadb34325a8b272a52e3ce45e4da2c2959eb81a0c8cb9a",
    "previousHash": "00000731a61c365f093fcbab8741d2ea29191c1da408dfcdd9332b568fe38cce",

```

Рисунок 6 – Процесс майнинга

Модификация

В качестве дальнейшего развития проекта были реализованы дополнительные возможности блокчейна. В систему была добавлена поддержка

транзакций между пользователями, а также реализована проверка авторства операций.

В ходе выполнения вариативной части были получены навыки:

- разработки многокомпонентных приложений на Java;
- работы с криптографическими алгоритмами;
- проектирования распределённых систем;
- тестирования и отладки программного обеспечения.

Листинг 2 – Класс блокчейна

```
public class PolyChain {

    public static ArrayList<Block> blockchain = new ArrayList<Block>();
    public static HashMap<String, TransactionOutput> UTXOs = new
HashMap<String, TransactionOutput>();

    public static int difficulty = 3;
    public static float minimumTransaction = 0.1f;
    public static Wallet walletA;
    public static Wallet walletB;
    public static Transaction genesisTransaction;

    public static void main(String[] args) {
        //add our blocks to the blockchain ArrayList:
        Security.addProvider(new
org.bouncycastle.jce.provider.BouncyCastleProvider()); //Setup Bouncycastle as a
Security Provider

        //Create wallets:
        walletA = new Wallet();
        walletB = new Wallet();
        Wallet coinbase = new Wallet();

        //create genesis transaction, which sends 100 NoobCoin to walletA:
        genesisTransaction = new Transaction(coinbase.publicKey,
walletA.publicKey, 100f, null);
        genesisTransaction.generateSignature(coinbase.privateKey);
        //manually sign the genesis transaction
        genesisTransaction.transactionId = "0"; //manually set the
transaction id
        genesisTransaction.outputs.add(new
TransactionOutput(genesisTransaction.recipient, genesisTransaction.value,
genesisTransaction.transactionId)); //manually add the Transactions Output
        UTXOs.put(genesisTransaction.outputs.get(0).id,
genesisTransaction.outputs.get(0)); //its important to store our first transaction
in the UTXOs list.

        System.out.println("Creating and Mining Genesis block... ");
        Block genesis = new Block("0");
        genesis.addTransaction(genesisTransaction);
        addBlock(genesis);

        //testing
    }
}
```

```

        Block block1 = new Block(genesis.hash);
        System.out.println("\nWalletA's balance is: " +
walletA.getBalance());
        System.out.println("\nWalletA is Attempting to send funds (40) to
WalletB...");
        block1.addTransaction(walletA.sendFunds(walletB.publicKey, 40f));
        addBlock(block1);
        System.out.println("\nWalletA's balance is: " +
walletA.getBalance());
        System.out.println("WalletB's balance is: " +
walletB.getBalance());

        Block block2 = new Block(block1.hash);
        System.out.println("\nWalletA Attempting to send more funds (1000)
than it has...");
        block2.addTransaction(walletA.sendFunds(walletB.publicKey, 1000f));
        addBlock(block2);
        System.out.println("\nWalletA's balance is: " +
walletA.getBalance());
        System.out.println("WalletB's balance is: " +
walletB.getBalance());

        Block block3 = new Block(block2.hash);
        System.out.println("\nWalletB is Attempting to send funds (20) to
WalletA...");
        block3.addTransaction(walletB.sendFunds(walletA.publicKey, 20));
        System.out.println("\nWalletA's balance is: " +
walletA.getBalance());
        System.out.println("WalletB's balance is: " +
walletB.getBalance());

        isChainValid();

    }

    public static Boolean isChainValid() {
        Block currentBlock;
        Block previousBlock;
        String hashTarget = new String(new char[difficulty]).replace('\0',
'0');

        HashMap<String, TransactionOutput> tempUTXOs = new
HashMap<String, TransactionOutput>(); //a temporary working list of unspent
transactions at a given block state.
        tempUTXOs.put(genesisTransaction.outputs.get(0).id,
genesisTransaction.outputs.get(0));

        //loop through blockchain to check hashes:
        for(int i=1; i < blockchain.size(); i++) {

            currentBlock = blockchain.get(i);
            previousBlock = blockchain.get(i-1);
            //compare registered hash and calculated hash:
            if(!currentBlock.hash.equals(currentBlock.calculateHash()))
        ){

            System.out.println("#Current Hashes not equal");
            return false;
        }
        //compare previous hash and registered previous hash

```

```

        if(!previousBlock.hash.equals(currentBlock.previousHash) )
    {
        System.out.println("#Previous Hashes not equal");
        return false;
    }
    //check if hash is solved
    if(!currentBlock.hash.substring( 0,
difficulty).equals(hashTarget)) {
        System.out.println("#This block hasn't been mined");
        return false;
    }

    //loop thru blockchains transactions:
    TransactionOutput tempOutput;
    for(int t=0; t <currentBlock.transactions.size(); t++) {
        Transaction currentTransaction =
currentBlock.transactions.get(t);

        if(!currentTransaction.verifySignature()) {
            System.out.println("#Signature on
Transaction(" + t + ") is Invalid");
            return false;
        }
        if(currentTransaction.getInputsValue() !=
currentTransaction.getOutputsValue()) {
            System.out.println("#Inputs are note equal
to outputs on Transaction(" + t + ")");
            return false;
        }

        for(TransactionInput input:
currentTransaction.inputs) {
            tempOutput =
tempUTXOs.get(input.transactionOutputId);

            if(tempOutput == null) {
                System.out.println("#Referenced
input on Transaction(" + t + ") is Missing");
                return false;
            }

            if(input.UTXO.value != tempOutput.value) {
                System.out.println("#Referenced
input Transaction(" + t + ") value is Invalid");
                return false;
            }

            tempUTXOs.remove(input.transactionOutputId);
        }

        for(TransactionOutput output:
currentTransaction.outputs) {
            tempUTXOs.put(output.id, output);
        }

        if( currentTransaction.outputs.get(0).recieipient !=
currentTransaction.recieipient) {

```

```

        System.out.println("#Transaction(" + t + ")
output recieipient is not who it should be");
        return false;
    }
    if( currentTransaction.outputs.get(1).recieipient !=
currentTransaction.sender) {
        System.out.println("#Transaction(" + t + ")
output 'change' is not sender.");
        return false;
    }

    }

    System.out.println("Blockchain is valid");
    return true;
}

public static void addBlock(Block newBlock) {
    newBlock.mineBlock(difficulty);
    blockchain.add(newBlock);
}
}

```

```

Creating and Mining Genesis block...
Transaction Successfully added to Block
Block Mined!!! : 000107236b351965f2734a3fb20053902ff4389d8a978bdc9e935d55e5966b36

WalletA's balance is: 100.0

WalletA is Attempting to send funds (40) to WalletB...
Transaction Successfully added to Block
Block Mined!!! : 00043602c474fa655ee99b4126c7828742cc2f3e66fa8b90184d1bc002f43456

WalletA's balance is: 60.0
WalletB's balance is: 40.0

WalletA Attempting to send more funds (1000) than it has...
#Not Enough funds to send transaction. Transaction Discarded.
Block Mined!!! : 000cb374154e23cf00f899818c37efae5592a761331be002b41c2b0c64f21b4a

WalletA's balance is: 60.0
WalletB's balance is: 40.0

WalletB is Attempting to send funds (20) to WalletA...
Transaction Successfully added to Block

WalletA's balance is: 80.0
WalletB's balance is: 20.0
Blockchain is valid

```

Рисунок 7 – Транзакции

ЗАКЛЮЧЕНИЕ

В результате проектной практики был получен практический опыт разработки и сопровождения программного обеспечения. Выполнение поставленных задач позволило закрепить знания в области программирования и систем контроля версий. Практика способствовала развитию навыков самостоятельной работы, ведения технической документации и использования современных инструментов разработки программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Документация Git. URL: <https://git-scm.com/doc> (дата обращения: 15.05.2026).
2. Документация Java. URL: <https://docs.oracle.com/javase/> (дата обращения: 15.05.2026).
3. Документация GitHub. URL: <https://docs.github.com/> (дата обращения: 15.05.2026).
4. Сатоши Накамото Bitcoin: A Peer-to-Peer Electronic Cash System. URL: <https://bitcoin.org/bitcoin.pdf> (дата обращения: 15.05.2026).
5. Документация OpenJDK. URL: <https://openjdk.org/> (дата обращения: 15.05.2025).